

Hands-On-Learning: Tuning LLM Decoding Parameters for Content Generation

Description

In this hands-on lab, you'll join a content automation startup and experiment with large language model (LLM) decoding parameters to optimize AI-generated text for different use cases. You'll systematically test temperature, top-k, top-p, and beam search settings to find the optimal balance between creativity and consistency for creative blogs, technical summaries, and customer support responses.

Learning Objectives

- Configure temperature, top-k, top-p, and beam search parameters in LLM APIs to control output diversity and coherence
- Compare deterministic (beam search) versus probabilistic (sampling) decoding strategies through systematic experimentation
- Select and justify appropriate parameter combinations for creative, technical, and customer-facing content use cases



- Evaluate model outputs qualitatively based on coherence, creativity, and task appropriateness
- Document parameter tuning methodology and reflect on trade-offs between creativity and consistency



Assignment Components

Introductory Slide

Title: Balancing AI Creativity and Consistency: Mastering LLM Decoding Parameters for Real-World Content Generation

Subtitle: Tune temperature, top-k, top-p, and beam search to optimize AI outputs for blogs, technical docs, and customer support

Tools to be used

Python 3.8+ with Hugging Face Transformers library

PyTorch or TensorFlow

Visual Studio Code (VS Code) with integrated terminal

GPT-2 model (via Hugging Face)

Pandas and Jupyter Notebook for data analysis

CSV and text file editors

Scenario

You've just joined a fast-growing content automation startup that uses large language models (LLMs) to generate marketing copy, technical summaries, and creative blog posts for global clients. The company's AI writing assistant has been receiving mixed feedback from clients:

- Marketing teams complain that blog introductions sound "robotic and boring"
- Technical documentation clients report outputs that are "too creative" and sometimes include invented specifications

- Customer support teams find responses either "too formal and cold" or "too casual and off-topic"

The VP of Product has tasked you with solving this problem. Your mission: experiment with LLM decoding parameters to bring the model's creativity and consistency into balance for each client scenario.

This is real work that AI engineers and prompt specialists perform daily at companies like Jasper, Copy.ai, and internal content teams at major corporations. These organizations spend significant engineering time tuning parameters to match brand voice, content type, and audience expectations.

You'll work with three realistic client scenarios:

1. Creative blog introductions - require personality and engagement (high creativity needed)
2. Technical product summaries - demand precision and accuracy (low randomness required)
3. Customer support responses - must stay helpful and on-topic (balanced control essential)

Your goal is to systematically test different parameter combinations, analyze their effects, and determine optimal configurations for each use case.

Instructions for Learners

You'll work with three test prompt sets provided in the repository files:

→ **creative_prompts.txt** - 5 prompts for blog introductions requiring engaging, varied language

→ **technical_prompts.txt** - 5 prompts for product descriptions requiring accuracy and clarity

→ **support_prompts.txt** - 5 prompts for customer service responses requiring helpfulness and consistency

These prompts simulate real client requests your startup receives across different industries.

ACTIVITY 1: Temperature Control - From Deterministic to Creative

Temperature is your primary dial for controlling creativity versus consistency. You'll explore how the same prompt produces dramatically different outputs at different temperature settings.

Step 1: Set up your development environment

- 1. Launch VS Code in your lab environment
- 2. Navigate to /module1_lab/ in the Explorer panel (left sidebar)
- 3. Open the starter file decoding_parameters_lab.py

- 
4. Verify you can see three prompt files: creative_prompts.txt, technical_prompts.txt, support_prompts.txt
 5. Open the integrated terminal (Terminal > New Terminal)
 6. Run the setup verification: python verify_setup.py
 7. Confirm Python 3.8+ and required libraries (transformers, torch) are installed

Expected outcome: You should see "Setup verification complete! All dependencies ready." in the terminal.

Step 2: Load the model and baseline configuration

- 
1. Locate the section marked # ACTIVITY 1: TEMPERATURE EXPERIMENTS in your starter file
 2. Review the model loading code that initializes GPT-2
 3. Run the setup code to load the model, tokenizer, and prompt files
 4. Observe the generate_text() function that accepts temperature, top-k, top-p, and beam parameters

└→ 5. Note how the function is documented - you'll create similar documentation for your deliverable

Expected outcome: Terminal displays "Setup complete! Model loaded and prompts ready." The model loads to your available device (CPU, CUDA, or MPS).

Tip: Model download takes 5-10 minutes on the first run (~500 MB). Subsequent runs use cached model.

Step 3: Experiment with temperature across all three use cases

- 1. Run the temperature sweep code in "Practice 1.1"
- 2. Test five temperature values: 0.2, 0.5, 0.7, 1.0, 1.5
- 3. Generate outputs for each temperature on all three use cases:
 - Creative blog introduction
 - Technical product summary
 - Customer support response

- 
4. Observe outputs in the terminal as they generate
 5. Review saved files in the outputs/ folder

Expected outcome: You'll have 15 output files (3 use cases × five temperatures). Notice how outputs change from rigid/repetitive at low temperatures to creative/chaotic at high temperatures.

What to observe:

- **Creative Blog (temp=0.2):** Does it sound engaging or flat and robotic?
- **Creative Blog (temp=1.0):** Is it creative and varied?
- **Creative Blog (temp=1.5):** Does it maintain coherence or become chaotic?
- **Technical Summary (temp=0.2):** Is it clear, factual, and professional?
- **Technical Summary (temp=0.7):** Does it add unnecessary creativity?
- **Technical Summary (temp=1.5):** Does it start "inventing" features?

- 
- **Customer Support (temp=0.5):** Is it helpful but too formal?
 - **Customer Support (temp=0.7):** Does it sound natural and reassuring?
 - **Customer Support (temp=1.2):** Does it become too casual or off-topic?

Step 4: Fine-tune and compare

1. Based on your observations, test intermediate temperature values in "Practice 1.2"
2. Use VS Code's comparison feature: Right-click a file > "Select for Compare", then right-click another > "Compare with Selected"
3. Take notes on which temperature ranges work best for each use case
4. Begin filling out observations for your Parameter Comparison Table (Deliverable 2)

Expected outcome: You've identified optimal temperature ranges for each use case and can articulate why certain temperatures work better.

Warning: Temperatures above 1.5 often produce incoherent gibberish.
Temperatures below 0.1 become extremely repetitive.

ACTIVITY 2: Vocabulary Control - Top-K vs Top-P Sampling

Now you'll explore how limiting vocabulary choices affects output quality. You'll compare two methods: top-k (fixed number of choices) and top-p (dynamic probability-based selection), and implement beam search for deterministic outputs.

Step 1: Experiment with top-k sampling

1. Navigate to # ACTIVITY 2: VOCABULARY CONTROL in your code file
2. Run "Practice 2.1" to test top-k values: 10, 50, 100

3. Keep the temperature constant at 0.7 to isolate top-k effects

4. Generate outputs for all three use cases with each top-k value

5. Save outputs to outputs/ folder with descriptive filenames

Expected outcome: 9 new files (3 use cases $\times$ 3 top-k values). Notice how vocabulary feels limited at $k=10$, more natural at $k=50$, and potentially unfocused at $k=100$.

What to observe:

At $k=10$: Does vocabulary feel too limited? Repetitive?

At $k=50$: Good balance of variety and focus?

At $k=100$: Does it maintain coherence or drift?

Step 2: Experiment with top-p (nucleus sampling)

1. Continue to "Practice 2.2" for top-p experiments

2. Test top-p values: 0.5, 0.9, 0.95

3. Disable top-k (set to 0) to isolate top-p effects

4. Generate outputs for all three use cases

5. Compare how top-p adapts dynamically versus top-k's fixed limit

Expected outcome: 9 more files showing how nucleus sampling produces more natural-sounding outputs by adapting vocabulary based on probability distribution.

What to observe:

- At $p=0.5$: Very focused but potentially repetitive?
- At $p=0.9$: Natural-sounding with good variety?
- At $p=0.95$: More creative but still coherent?

Step 3: Implement beam search (REQUIRED for Deliverable 3)

1. Navigate to "Practice 2.2a: Beam Search Experiments"
2. Create the `generate_text_beam()` function for deterministic decoding

3. Test beam search with num_beams = 3 and num_beams=5

4. Use the same prompt for both tests to enable fair comparison

5. Save both outputs to submission/beam_search_comparison.txt

6. Add analysis notes comparing beam search to sampling methods

Expected outcome: **Deliverable three complete** - You have a file with clearly labeled beam=3 and beam=5 outputs showing how higher beam values produce more coherent, deterministic text.

Tip: Beam search is deterministic (same input = same output), while sampling is probabilistic (outputs vary each run).

ACTIVITY 3: Creating Required Deliverables

Now you'll synthesize your findings and create the remaining deliverables for AI grading.

Step 1: Create Model Setup Code - Deliverable 1

1. Navigate to "Practice 3.1: Create Model Setup Code"
2. Review the complete `generate_text()` function that handles both sampling and beam search
3. Ensure the function includes proper documentation with parameter descriptions
4. Save the code to `submission/model_setup.py`
5. Test the function with sample inputs to verify it runs without errors

Expected outcome: **Deliverable one complete** - A well-documented Python file containing your text generation function with adjustable parameters (temperature, top-k, top-p, beam).

Step 2: Create Parameter Comparison Table - Deliverable 2

1. Navigate to "Practice 3.2: Create Parameter Comparison Table"
2. Define at least three different parameter configurations based on your experiments
3. Customize the temperature, top_k, and top_p values based on what worked best

4. Generate fresh outputs for each configuration

5. Save results to submission/parameter_comparison.csv

6. Verify the CSV includes all required columns: configuration_id, temperature, top_k, top_p, beam, use_case, prompt, output_sample

Expected outcome: **Deliverable two complete** - A CSV file with at least three unique parameter sets showing meaningful variation and substantive output samples (>50 tokens each).

Tip: Choose configurations that clearly demonstrate different behaviors—don't use similar values.

Step 3: Create Use-Case Summary Table - Deliverable 4

1. Navigate to "Practice 3.3: Create Use-Case Summary Table"

2. Define your optimal configuration for each use case:

→ Creative Blog

→ Technical Summary

→ Customer Support

IMPORTANT: Adjust the default values based on YOUR experimental findings

4. Write a detailed rationale for each configuration, explaining your parameter choices

5. Save to submission/use_case_summary.csv

Expected outcome: **Deliverable four complete** - A CSV with optimal parameters and thoughtful rationale for all three scenarios, demonstrating understanding of parameter effects.

Your rationale should address:

- Why this temperature range is appropriate
- How top-p/top-k choice affects vocabulary
- What trade-offs are you making
- How this serves the use case requirements

Step 4: Write Reflection Report - Deliverable 5

1. Navigate to "Practice 3.4: Write Reflection Report"
2. Open the template saved to submission/reflection_report.txt
3. **REPLACE the template** with your own 250-300-word reflection
4. Address all required elements:
 - Specific tuning choices you made
 - Trade-offs between creativity and consistency

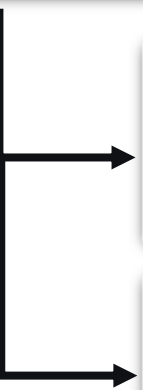


Key insights about parameter impacts

Lessons applicable to production systems

Expected outcome: **Deliverable five complete** - A 250-300 word reflection demonstrating a clear understanding of parameter tuning methodology and real-world applications.

Your reflection should answer:



What temperature ranges work best for each use case and why?

How do top-k and top-p differ in practice?



Which parameter has the most significant impact on quality?

What surprised you during experimentation?

How would you approach parameter tuning in production?

Warning: The template is a guide only. AI grading will check that you've written original content based on your actual experiments.

Step 5: Validate all deliverables

1. Run "Practice 3.5: Validate all deliverables before submission"

2. Confirm all five files are present in /submission/ folder:

→ model_setup.py

→ parameter_comparison.csv

→ beam_search_comparison.txt

→ use_case_summary.csv

└→ reflection_report.txt

3. Verify file sizes are reasonable (not empty)

4. Review each file one final time for completeness

5. Click the "Submit Project" button to upload for AI grading

Expected outcome: Validation script confirms " ALL DELIVERABLES COMPLETE!" and you're ready to submit.

Deliverable for Submission (AI Grading)

1. Deliverable 1: Model Setup Code

Question Prompt: "Submit your Python code file containing the text generation function that implements adjustable decoding parameters (temperature, top-k, top-p, and beam search). Your function should be well-documented with comments explaining each parameter's purpose and should run without errors when tested with sample prompts."

Response Type: File Upload

File Types: Coding (.py, .txt, .md)

File Name: model_setup.py

Grading Criteria:

- Function accepts all required parameters (temperature, top-k, top-p, beam) - 5 pts
- Code runs without errors - 5 pts
- Successfully generates text with different parameter combinations - 5 pts
- Code includes clear documentation and comments - 5 pts

2. Deliverable 2: Parameter Comparison Table

Question Prompt: "Submit a CSV file containing at least three unique parameter configurations you tested, along with corresponding output samples. Each configuration should demonstrate meaningful variation in the parameters (temperature, top-k, top-p) and include the generated text, the use case, and the prompt used. Your table should clearly show how different parameter combinations affect output quality."

Response Type: File Upload

File Types: Text (.csv, .txt), Coding (.md for formatted tables)

File Name: parameter_comparison.csv

Grading Criteria:

- Contains at least three different parameter combinations - 5 pts
- Includes all required columns (configuration_id, temperature, top_k, top_p, beam, use_case, prompt, output_sample) - 5 pts
- Output samples are substantive (>50 tokens each) - 5 pts
- Parameters show meaningful variation (not all similar values) - 5 pts

3. Deliverable 3: Beam Search Comparison Analysis

Question Prompt: "Submit a text file containing two clearly labeled outputs generated using beam search with beam=3 and beam=5. Use the same prompt for both outputs to enable fair comparison. Include brief analysis notes explaining how the outputs differ and how beam search compares to sampling-based methods. What advantages does beam search offer for specific use cases?"

Response Type: File Upload

File Types: Text (.txt, .md), Audio (.mp3, .wav for verbal explanation), Video (.mp4, .mov for screen recording with explanation)

→ **File Name:** beam_search_comparison.txt (or equivalent for audio/video)

Grading Criteria:

→ Contains output with beam=3 clearly labeled - 5 pts

→ Contains output with beam=5 clearly labeled - 5 pts

→ Uses the same prompt for both outputs for fair comparison - 5 pts

4. Deliverable 4: Use-Case Summary and Rationale

Question Prompt: "Submit a CSV file or structured document containing your optimal parameter configuration for all three use cases (Creative Blog, Technical Summary, Customer Support). For each use case, specify your chosen temperature, top-k, and top-p values and provide a detailed rationale explaining WHY these parameters are appropriate. Your rationale should demonstrate understanding of how each parameter affects output quality and why your choices serve each use case's unique requirements."

Response Type: File Upload

File Types: Text (.csv, .txt, .md), Audio (.mp3, .wav for verbal explanation), Video (.mp4, .mov for presentation)

→ **File Name:** use_case_summary.csv (or equivalent)

Grading Criteria:

→ Contains configurations for all three use cases - 6 pts

→ Temperature values are appropriate for each use case - 6 pts

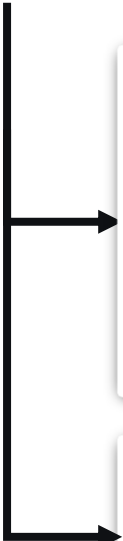
→ Detailed rationale is provided for each configuration - 7 pts

→ Rationale demonstrates clear understanding of parameter effects - 6 pts

5. Deliverable 5: Reflection Report on Parameter Tuning

Question Prompt: "Write a 250–300-word reflection explaining your parameter tuning methodology and key learnings. Address the following: What specific tuning choices did you make and why? How did you balance the trade-off between creativity and consistency for different use cases? What insights did you gain about how temperature, top-k, and top-p impact output quality? Which parameter had the biggest effect? What would you do differently in a production environment? Your reflection should demonstrate a clear understanding of parameter tuning concepts and their real-world applications."

Response Type: File Upload



File Types: Text (.txt, .md), Audio (.mp3, .wav, .aac, .flac for verbal reflection), Video (.mp4, .mov, .webm for recorded reflection - max 15 min)

File Name: reflection_report.txt (or equivalent)

Grading Criteria:

- Meets word count requirement (250-300 words for text; equivalent depth for audio/video) - 4 pts
- Explains specific tuning choices with concrete examples - 5 pts
- Discusses trade-offs between creativity and consistency - 5 pts
- Demonstrates clear understanding of parameter impacts - 6 pts

Additional Information

Best Practices for

AI-Graded Submission

Creating Effective Question Prompts

1. Be Specific: Include specific numbers or criteria in your prompts to help AI generate clear assessment criteria.
2. Include Context: Reference specific elements from the HOL activity.
3. Encourage Depth: Ask for examples, justifications, or connections.
4. Align with Objectives: Ensure prompts assess stated learning objectives.

Submission Requirement Guidelines

- 
1. Clarity: Each requirement should have one clear focus.
 2. Balance: Mix different types of submissions (creative, analytical, reflective).
 3. Inclusivity: Always provide alternative formats.
 4. Progression: Order requirements from concrete to abstract
- Videos: 15 minutes maximum

Accessibility Considerations:

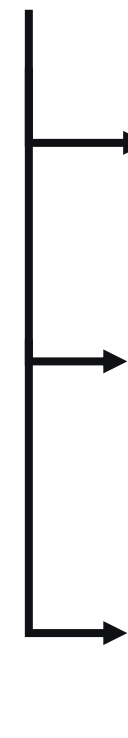
- Always allow multiple file types for each requirement.
- Include both video AND audio options for verbal responses.
- Provide text alternatives for visual submissions.

Quality Assurance Checklist

Before finalizing your HOL:

- All prompts are clear and specific

- 
- Multiple file types offered for each upload requirement
 - Submission types match the nature of the task
 - Requirements align with learning objectives
 - Instructions are step-by-step and actionable
 - Accessibility options are available throughout

- 
- Time estimates are realistic and add up to the stated duration
 - Expected outcomes are provided for each major step
 - Tips and warnings are included where necessary

APPENDIX: Supporting Materials

Prompt Files Content

creative_prompts.txt:

The future of artificial intelligence in healthcare is

Write an engaging blog introduction about sustainable fashion and why it matters in 2025

In a world where technology connects us all, the most important skill we can develop is

The secret to building a successful startup isn't what most people think. It's actually
Picture this: You wake up one morning to discover that time travel has just been invented. The first thing you'd want to do is

technical_prompts.txt:

This cloud-based data analytics platform offers real-time processing capabilities with

The new API endpoint provides RESTful access to user authentication services, supporting

Our enterprise software solution integrates seamlessly with existing infrastructure to deliver

The neural network architecture implements a transformer-based model with attention mechanisms that

This cybersecurity framework employs multi-layer encryption protocols to protect sensitive data through

support_prompts.txt:

I'm having trouble logging into my account. I've tried resetting my password, but haven't received the email.

The product I ordered arrived damaged. What should I do to get a replacement or refund?

Can you explain how the monthly subscription billing works and when I'll be charged?

I'd like to upgrade my plan to access premium features. What are my options, and how do I proceed?

My payment was declined, but the amount was deducted from my account. How can this be resolved?

Common Issues & Solutions

Problem: All outputs sound the same regardless of parameters. Solution: Verify that `do_sample=True` is set in your `generate_text` function. Check that parameters are actually being passed to `model.generate()`. Run: `python -c "import torch; print(torch.__version__)"` to verify PyTorch.

Problem: Model generates repetitive text (same phrases over and over). Solution: The `no_repeat_ngram_size=2` parameter is already included. Try increasing it to 3 if repetition persists. Consider slightly increasing the temperature or top-p.

Problem: Temperature > 1.0 produces gibberish. Solution: This is expected behavior—very high temperatures break coherence. Stay between 0.8-1.2 for creative tasks. Combine high temperature with moderate top-p (e.g., temp=1.2, top-p=0.85).

Problem: Outputs are too short. Solution: Increase the max_length parameter (try 200-250). The min_length=50 should prevent very short outputs. Some prompts naturally lead to shorter responses.

Problem: Can't decide between top-k and top-p. Solution: When in doubt, use top-p=0.9 as a default. Top-p adapts better to context than fixed top-k. Modern LLM applications primarily use top-p.

Problem: VS Code is not showing output files. Solution: Click the refresh icon in the Explorer panel. Ensure the outputs/ and submission/ folders exist (script creates them automatically). Check the terminal for any error messages during file writing.

Problem: Results don't match between runs. Solution: This is normal for sampling-based generation (`do_sample=True`). Set a random seed for reproducibility: `torch.manual_seed(42)`. For truly deterministic outputs, use beam search.

Problem: Model download fails or is very slow. Solution: First download will take 5-10 minutes (~500MB). Check the internet connection. If a timeout occurs, restart, and the download will resume. Alternatively, use a smaller model like "distilgpt2".

Problem: Deliverable files not being created. Solution: Ensure the submission/ folder exists (create it if needed: `os.makedirs('submission', exist_ok=True)`). Check file permissions in your workspace. Verify you're running the code in the correct directory. Look for error messages in the terminal output.

Key Takeaways

Temperature Control:

Low (0.2-0.5): Deterministic, factual, consistent—ideal for technical/support content



Medium (0.6-0.8): Balanced, natural-sounding—good for general-purpose applications

High (0.9-1.5): Creative, varied, unpredictable—best for creative content

Vocabulary Filtering:



Top-k sets hard vocabulary boundaries: Simple and predictable but can feel rigid

Top-p (nucleus sampling) adapts dynamically: Adjusts vocabulary based on probability distribution, industry standard for most modern applications (sweet spot: 0.85-0.95)

Parameter Interactions:

- Temperature and top-p work together multiplicatively
- High temperature + low top-p = creative but focused
- Low temperature + high top-p = consistent but allows some variety
- Both at extremes = usually problematic

Use Case Specificity:

- No universal "best" configuration exists

- 
- Creative tasks need higher temperature and top-p
 - Technical/factual tasks need lower temperature
 - Customer-facing tasks need balance
 - Always validate on your specific domain

Beam Search vs Sampling:

→ Beam search (deterministic): Best for technical content requiring consistency, explores multiple paths simultaneously

→ Sampling (probabilistic): More natural-sounding for creative/conversational content, allows for greater diversity

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.